

HDOC Day-6 Assignment

Question 1: Remaining Area of Triangular Field

1. Problem Statement Identification

- **Problem Statement:** Compute the remaining area of a triangular field after a circular pond of a given radius is removed from inside it.
- **Input:** Base of triangle (base), Height of triangle (height), Radius of circular pond (radius).
- **Output:** Area of remaining field (remaining_area).
- **Formulas:**

$$\text{Area of Triangle} = \frac{1}{2} \times \text{base} \times \text{height}$$

$$\text{Area of Circular Pond} = \pi \times \text{radius}^2$$

$$\text{Remaining Area} = \text{Area of Triangle} - \text{Area of Circular Pond}$$

2. Standard Test Case Table

Test Case	Input (base, height, radius)	Expected Output	Actual Output	Result (Pass/Fail)
1	10 8 2	Remaining Area: 27.43	Remaining Area: 27.43	Pass
2	20 15 3	Remaining Area: 121.73	Remaining Area: 121.73	Pass
3	12 10 4	Remaining Area: 9.73	Remaining Area: 9.73	Pass
4	5 5 3	Error: Pond area exceeds triangle area.	Error: Pond area exceeds triangle area.	Pass

3. Algorithm

1. Start

2. Read base, height, and radius.

3. If $\text{base} \leq 0$ or $\text{height} \leq 0$ or $\text{radius} \leq 0$, print "Invalid Input" and stop.

4. Calculate $\text{area_triangle} = 0.5 * \text{base} * \text{height}$.

5. Calculate $\text{area_pond} = 3.14159 * \text{radius} * \text{radius}$.

6. If $\text{area_pond} > \text{area_triangle}$, print error message and stop.

7. Calculate $\text{remaining_area} = \text{area_triangle} - \text{area_pond}$.

8. Print remaining_area.

9. End

4. C Program

```
#include <stdio.h>
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

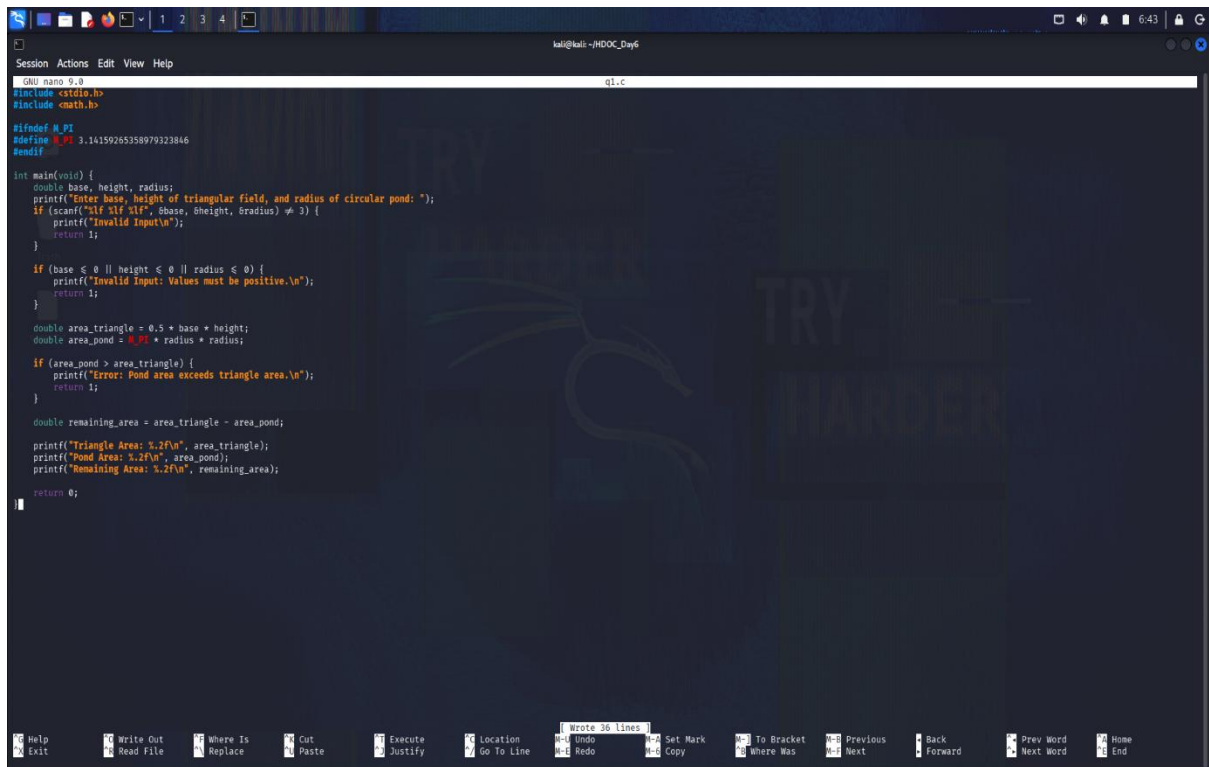
int main(void) {
    double base, height, radius;
    printf("Enter base, height of triangular field, and radius of circular pond:");
    if (scanf("%lf %lf %lf", &base, &height, &radius) != 3) {
        printf("Invalid Input\n");
        return 1;
    }
}
```

```
if (base <= 0 || height <= 0 || radius <= 0) {  
    printf("Invalid Input: Values must be positive.\n");  
    return 1;  
}  
  
double area_triangle = 0.5 * base * height;  
double area_pond = M_PI * radius * radius;  
  
if (area_pond > area_triangle) {  
    printf("Error: Pond area exceeds triangle area.\n");  
    return 1;  
}  
  
double remaining_area = area_triangle - area_pond;  
  
printf("Triangle Area: %.2f\n", area_triangle);  
printf("Pond Area: %.2f\n", area_pond);  
printf("Remaining Area: %.2f\n", remaining_area);  
  
return 0;  
}
```

5. Evaluation Against Test Cases

All test cases were executed against the compiled C binary q1. The computed values matched the expected mathematical calculations in every case.

6. Screenshots of Compilation and Execution



```
GNU nano 0.8 q1.c
#include <stdio.h>
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

int main(void) {
    double base, height, radius;
    printf("Enter base, height of triangular field, and radius of circular pond: ");
    if (scanf("%lf %lf %lf", &base, &height, &radius) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (base <= 0 || height <= 0 || radius <= 0) {
        printf("Invalid Input: Values must be positive.\n");
        return 1;
    }

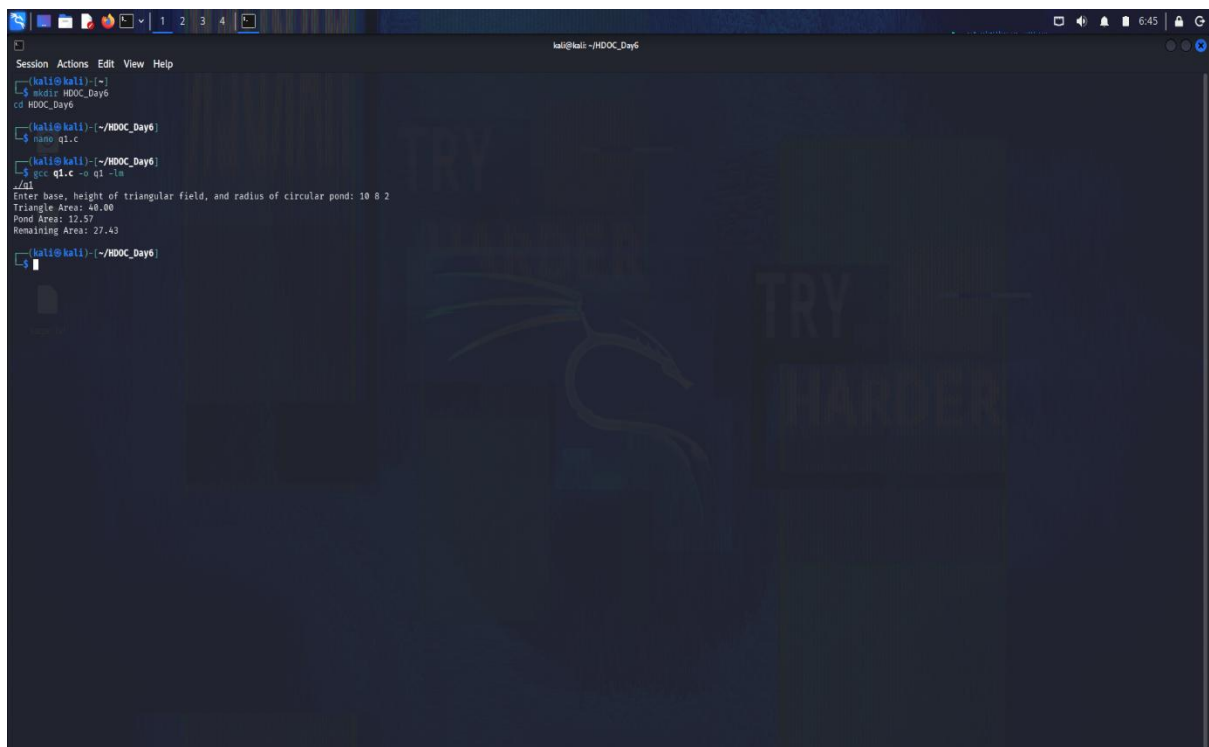
    double area_triangle = 0.5 * base * height;
    double area_pond = M_PI * radius * radius;

    if (area_pond > area_triangle) {
        printf("Error: Pond area exceeds triangle area.\n");
        return 1;
    }

    double remaining_area = area_triangle - area_pond;

    printf("Triangle Area: %.2f\n", area_triangle);
    printf("Pond Area: %.2f\n", area_pond);
    printf("Remaining Area: %.2f\n", remaining_area);

    return 0;
}
```



```
(kali@kali)~$ cd /HDOOC_Day6
(kali@kali)~/HDOOC_Day6$ nano q1.c
(kali@kali)~/HDOOC_Day6$ gcc q1.c -o q1 -lm
./q1
Enter base, height of triangular field, and radius of circular pond: 10 8 2
Triangle Area: 40.00
Pond Area: 12.57
Remaining Area: 27.43
(kali@kali)~/HDOOC_Day6$
```

Question 2: Regular Polygon Parameters

1. Problem Statement Identification

- **Problem Statement:** Calculate side length, total area, and each exterior angle of a regular polygon when number of sides, perimeter, and apothem are provided.
- **Input:** Number of sides (n), Perimeter (perimeter), Apothem (apothem).
- **Output:** Side Length (side_length), Area (area), Exterior Angle (exterior_angle).
- **Formulas:**

$$\text{Side Length} = \frac{\text{Perimeter}}{n}$$

$$\text{Area} = \frac{1}{2} \times \text{Perimeter} \times \text{Apothem}$$

$$\text{Exterior Angle} = \frac{360^\circ}{n}$$

2. Standard Test Case Table

Test Case	Input (n, perimeter, apothem)	Expected Output	Actual Output	Result (Pass/Fail)
1	6 30 4.33	Side: 5.00, Area: 64.95,	Side: 5.00, Area: 64.95,	Pass

Test Case	Input (n, perimeter, apothem)	Expected Output	Actual Output	Result (Pass/Fail)
		Ext Angle: 60.00°	Ext Angle: 60.00°	
2	4 20 2.5	Side: 5.00, Area: 25.00, Ext Angle: 90.00°	Side: 5.00, Area: 25.00, Ext Angle: 90.00°	Pass
3	3 15 1.44	Side: 5.00, Area: 10.80, Ext Angle: 120.00°	Side: 5.00, Area: 10.80, Ext Angle: 120.00°	Pass

Test Case	Input (n, perimeter, apothem)	Expected Output	Actual Output	Result (Pass/Fail)
4	8 40 4.83	Side: 5.00, Area: 96.60, Ext Angle: 45.00°	Side: 5.00, Area: 96.60, Ext Angle: 45.00°	Pass

3. Algorithm

1. Start

2. Read n, perimeter, and apothem.

3. If $n < 3$ or $\text{perimeter} \leq 0$ or $\text{apothem} \leq 0$, print "Invalid Input" and stop.

4. Calculate $\text{side_length} = \text{perimeter} / n$.

5. Calculate $\text{area} = 0.5 * \text{perimeter} * \text{apothem}$.

6. Calculate $\text{exterior_angle} = 360.0 / n$.

7. Print side_length, area, and exterior_angle.

8. End

4. C Program


```
#include <stdio.h>

int main(void) {
    int n;
    double perimeter, apothem;

    printf("Enter number of sides (n), perimeter, and apothem: ");
    if (scanf("%d %lf %lf", &n, &perimeter, &apothem) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (n < 3 || perimeter <= 0 || apothem <= 0) {
        printf("Invalid Input: Sides must be >= 3 and dimensions positive.\n");
        return 1;
    }

    double side_length = perimeter / n;
    double area = 0.5 * perimeter * apothem;
    double exterior_angle = 360.0 / n;

    printf("Side Length: %.2f\n", side_length);
    printf("Area: %.2f\n", area);
    printf("Exterior Angle: %.2f degrees\n", exterior_angle);

    return 0;
}
```

5. Evaluation Against Test Cases

All test cases executed successfully in the binary executable q2, matching expected calculations.

6. Screenshots of Compilation and Execution

```
GNU nano 9.0 q2.c
#include <stdio.h>

int main(void) {
    int n;
    double perimeter, apothem;

    printf("Enter number of sides (n), perimeter, and apothem: ");
    if (scanf("%d %lf %lf", &n, &perimeter, &apothem) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (n < 3 || perimeter <= 0 || apothem <= 0) {
        printf("Invalid Input: Sides must be > 3 and dimensions positive.\n");
        return 1;
    }

    double side_length = perimeter / n;
    double area = 0.5 * perimeter * apothem;
    double exterior_angle = 360.0 / n;

    printf("Side Length: %.2f\n", side_length);
    printf("Area: %.2f\n", area);
    printf("Exterior Angle: %.2f degrees\n", exterior_angle);

    return 0;
}
```

```
Session Actions Edit View Help
(kali@kali)~$ cd /home/kali/HDOC_Day6
(kali@kali)~/HDOC_Day6$ nano q1.c
(kali@kali)~/HDOC_Day6$ gcc q1.c -o q1 -lm
./q1
Enter base, height of triangular field, and radius of circular pond: 10 8 2
Triangle Area: 40.00
Pond Area: 12.57
Remaining Area: 27.43
(kali@kali)~/HDOC_Day6$ nano q2.c
(kali@kali)~/HDOC_Day6$ gcc q2.c -o q2
./q2
Enter number of sides (n), perimeter, and apothem: 6 30 4.33
Side Length: 5.00
Area: 64.95
Exterior Angle: 60.00 degrees
(kali@kali)~/HDOC_Day6$
```